GOLD: Synchronization (Fri Oct 16)

The Crime Scene

Consider two workers with one shared `static` counter.

Each worker adds +1 a million times.

Expected counter value: 2 million, right?

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new RaceCondition());
        Thread t2 = new Thread(new RaceCondition());
        long start = System.currentTimeMillis();
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        long ms = System.currentTimeMillis() - start;
        IO.println("expected 2000000, got " + RaceCondition.counter + "  (" + ms + " m
   }
}

class RaceCondition implements Runnable {
    static int counter = 0;                         // one shared box

    @Override
    public void run() {
        for (int i = 0; i < 1_000_000; i++) {
            counter++;
        }
    }
}
```

Some outputs:

```
expected 2000000, got 1903349  (8 ms)
expected 2000000, got 1980318  (10 ms)
expected 2000000, got 1866781  (5 ms)
expected 2000000, got 1066736  (7 ms)
expected 2000000, got 1277138  (13 ms)
```

Huh?

Run it repeatedly, and we (almost) never get 2,000,000 exactly.

Today we'll learn

- why this happens, and
- how we can prevent this

Race conditions

`counter++` is a lie

Let's take a closer look at the task:

```
public void run() {
    for (int i = 0; i < 1_000_000; i++)
        counter++;
}
```

It looks pretty harmless: `counter++` in one step.

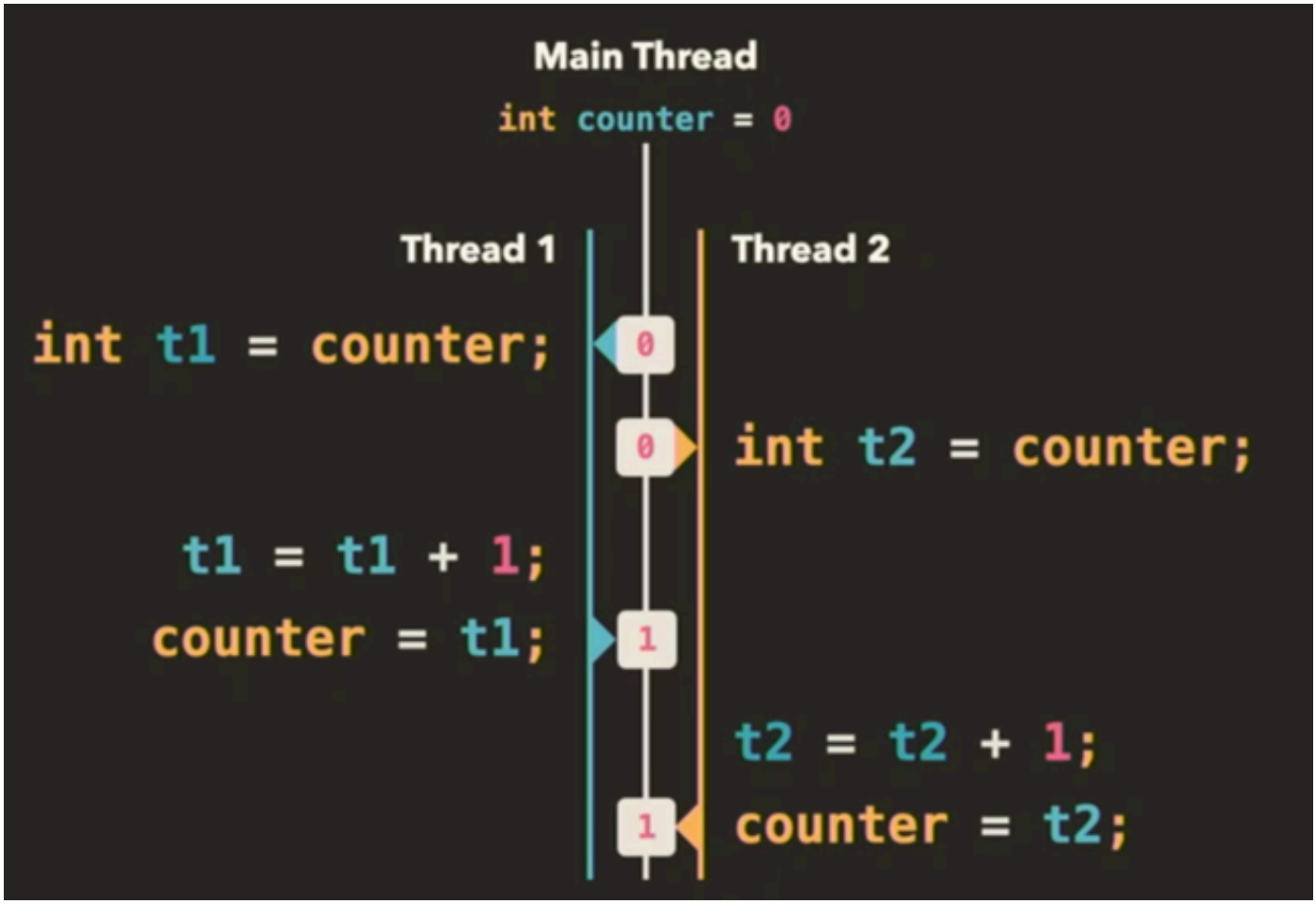
However, the machine runs three:

```
int t = counter;      // 1. READ  the current value
t = t + 1;            // 2. ADD   one to my private copy
counter = t;          // 3. WRITE my copy back
```

- **Within** one thread: these three always run in order
- **Between** threads: the scheduler can interleave them at this granularity
    - a thread can be interrupted between step 1 and step 3
    - in general, we have no control of how much can happen in other threads between two steps in our thread

One line of Java is often not one indivisible action.

Let's see how this explains where the missing counter increments went:



Both threads read 0, and both wrote 1.

Both increments happened, but the first one got blown away by the second.

Remember: When Thread 2 doesn't see Thread 1's work, it can overwrite it with its own stale answer.

This is a **race condition**: code where the answer depends on which thread wins the "race" and how different instructions got interleaved.

The Fix: synchronized

One at a time, please

Summary so far: race conditions can happen when we combine

- static fields or shared object references with
- concurrency

The fix is to use Java's keyword **`synchronized`**.

`synchronized` is a block structure that allows two or more threads to avoid race conditions.

Syntax:

```
synchronized (someObject) {
  statements; // modify shared data here
}
```

Among all threads running synchronizing using the same object, only one thread can be “inside” the block of statements at a time. The rest wait outside at the gate.

Above `someObject` is an actual Java object serving as the lock.

Intuition: think of a lockable door:

- A thread that arrives at `synchronized` checks whether the door is locked
- If the door is unlocked, the thread goes in and locks the door behind it.
- When it is finished with the block, it unlocks the door and leaves.
- If the door is locked when a thread arrives, the thread waits for the door to be unlocked.

Here is the counter again, with the gate in place. Run it:

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new NoRaceCondition());
        Thread t2 = new Thread(new NoRaceCondition());
        long start = System.currentTimeMillis();
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        long ms = System.currentTimeMillis() - start;
        IO.println("expected 2000000, got " + NoRaceCondition.counter + "  (" + ms + "
    }
}

class NoRaceCondition implements Runnable {
    static int counter = 0;
    static Object gateKeeper = new Object();       // one gate for everyone: static

    @Override
    public void run() {
        for (int i = 0; i < 1_000_000; i++) {
            synchronized (gateKeeper) {
                counter++;                         // the whole read-add-write, undist
            }
        }
    }
}
```

Code for the fixed task:

```
private static int counter = 0;
private static Object gateKeeper = new Object();

public void run() {
    for (int i = 0; i < 1_000_000; i++) {
        synchronized (gateKeeper) {
            counter++;                 // the whole read-add-write, undisturbed
        }
    }
}
```

Using a **static** object for the gatekeeper here is the right choice:

- The danger comes from two threads sharing a **static field** counter

- even though each thread has a **different** RaceCondition object
- So we need a `static gateKeeper` so that both threads see the same lock.

Watch out: using a non-static Object gatekeeper would mean that each thread sees a different lock and the race still happens!

This Is Your Paycheck

This is your paycheck

You've done so well in your first six months as a software engineer that both your supervisor and the head of engineering decide to give you a raise immediately after the performance review:

A shared `Employee` object has a `salary` of 40.

The supervisor's thread runs `salary += 5`; the head of engineering's thread runs `salary += 10`.

Each `+=` is really three steps as we have seen: read, add, write. No gate anywhere.

```
salary = 40

Supervisor's thread:            Head of Engineering's thread:
    int t = salary;                 int u = salary;
    t = t + 5;                      u = u + 10;
    salary = t;                     salary = u;
```

The six steps can interleave in any order that keeps each thread's own three in sequence.

Pause and predict (poll time)

Q. Which final values of `salary` are possible?

- **A.** Only `55`
- **B.** `45`, `50` or `55`
- **C.** `45` or `50` only
- **D.** `40`, `45`, `50` or `55`

Whose Gate?

Whose gate?

Let's make one change: the gate is an ordinary field, `Object gate = new Object();`, not `static`.

```
class Main {
    void main() throws InterruptedException {
        Thread t1 = new Thread(new Counter());
        Thread t2 = new Thread(new Counter());
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        IO.println("expected 2000000, got " + Counter.counter);
    }
}

class Counter implements Runnable {
    static int counter = 0;
    Object gate = new Object();                    // NOT static: every worker gets it

    @Override
    public void run() {
        for (int i = 0; i < 1_000_000; i++) {
            synchronized (gate) {
                counter++;
            }
        }
    }
}
```

Pause and predict (poll time)

Q. What does it print?

- **A.** `2000000`
- **B.** The race is still there
- **C.** Compile error: the object in `synchronized (...)` must be `static`
- **D.** Deadlock: each thread waits for the other's gate

Costs of Synchronization

How much overhead?

We included timing information for the counter example for a reason:

Version	Result	Time (typical Ed run)
`RaceCondition` (no gate)	**wrong**: `1085846`-style garbage	~10 ms
`NoRaceCondition` (gate)	**right**: `2000000` exact	~100 ms

The lock costs about 10x on this loop.

- Threads queue at the gate: guarded code runs one-at-a-time.
- The lock isn't free: Java has to do a thread-safe write every time we update `gatekeeper` to record locked or released.
- Each write could cost ~ 10 ns, but this adds up over 2 million actions

Locking is mandatory for correctness, despite the costs.

We want to keep locking as fine grained as possible to protect shared data while allowing unshared work in parallel.

- So: guard only the shared touch(es) of static fields and shared references
- keep the locked region small
- prefer designs that share nothing if possible (e.g. yesterday's prime number counters needed no gate at all).

Measuring the Costs

Let's experiment with different versions of the two-worker counter:

- no gate;
- the gate **inside** the loop, around each `counter++`;
- and the gate **around** the whole loop.

```
class Main {
    static int counter = 0;
    static final Object gate = new Object();
    static final int N = 1_000_000;

    void main() throws InterruptedException {
        time("racy (no gate)", new Racy());
        time("fine (gate inside the loop)", new Fine());
        time("coarse (gate around the loop)", new Coarse());
    }

    static void time(String label, Runnable job) throws InterruptedException {
        counter = 0;
        Thread t1 = new Thread(job), t2 = new Thread(job);
        long start = System.nanoTime();
        t1.start(); t2.start();
        t1.join(); t2.join();
        long ms = (System.nanoTime() - start) / 1_000_000;
        IO.println(String.format("%-30s got %8d in %4d ms", label, counter, ms));
    }
}

class Racy implements Runnable {
    public void run() { for (int i = 0; i < Main.N; i++) Main.counter++; }
}

class Fine implements Runnable {
    public void run() { for (int i = 0; i < Main.N; i++) { synchronized (Main.gate) {
}

class Coarse implements Runnable {
    public void run() { synchronized (Main.gate) { for (int i = 0; i < Main.N; i++) Ma
}
```

```
racy (no gate)                got  1039469 in    7 ms
fine (gate inside the loop)   got  2000000 in   92 ms
coarse (gate around the loop) got  2000000 in    4 ms
```

The gate around the loop is correct *and* fast, because it is taken twice instead of two million times.

It is also not parallel at all: the second worker waits for the first one's entire loop.

This is fine when the loop is the whole job; it is wrong when the workers also have unshared work to do.

The rule is: guard the shared touches, and let the rest run in parallel.

A Tempting Gate

A tempting gate

Two classes that have nothing to do with each other each guard their own work with a string literal, because it was handy:

```
class A implements Runnable {
    public void run() {
        synchronized ("lock") { work for 400 ms }
    }
}

class B implements Runnable {                    // written by someone else, in anothe
    public void run() {
        synchronized ("lock") { work for 400 ms }
    }
}

// main starts one A and one B at the same moment, then joins both
```

Pause and predict (poll time)

Q. `A` and `B` run at the same time. How long until both are done?

- **A.** About 400 ms
- **B.** About 800 ms
- **C.** Compile error: a `String` cannot be a lock
- **D.** Deadlock

Gatekeeping rules

Rules of the gate

1. **Same object, or no protection.**
    - Two threads are safe from each other *only if they synchronize on the same object.*
    - Two different gateKeepers = two doors into the same room.
2. **Make a dedicated gate.**
    - `private static Object gateKeeper = new Object();`
    - This is an object whose only job is being the lock.
    - Whether or not the gate should be static depends on your program.
    - If the gateKeeper object has other duties, you have to make sure it never gets reassigned (replaces the lock).
3. **Don't use a `String` literal or a boxed number as the gate.**
    - Java pools and caches those objects behind the scenes
    - you may be unknowingly sharing your gate with strangers elsewhere in the program
    - Three different synchronized blocks in three classes all using `"cat"` will have one common lock
    - Unnecessary lock sharing isn't dangerous, but reduced concurrency

Find the Student

Example: Find the Student

The registrar has a problem: student records live on three different file servers.

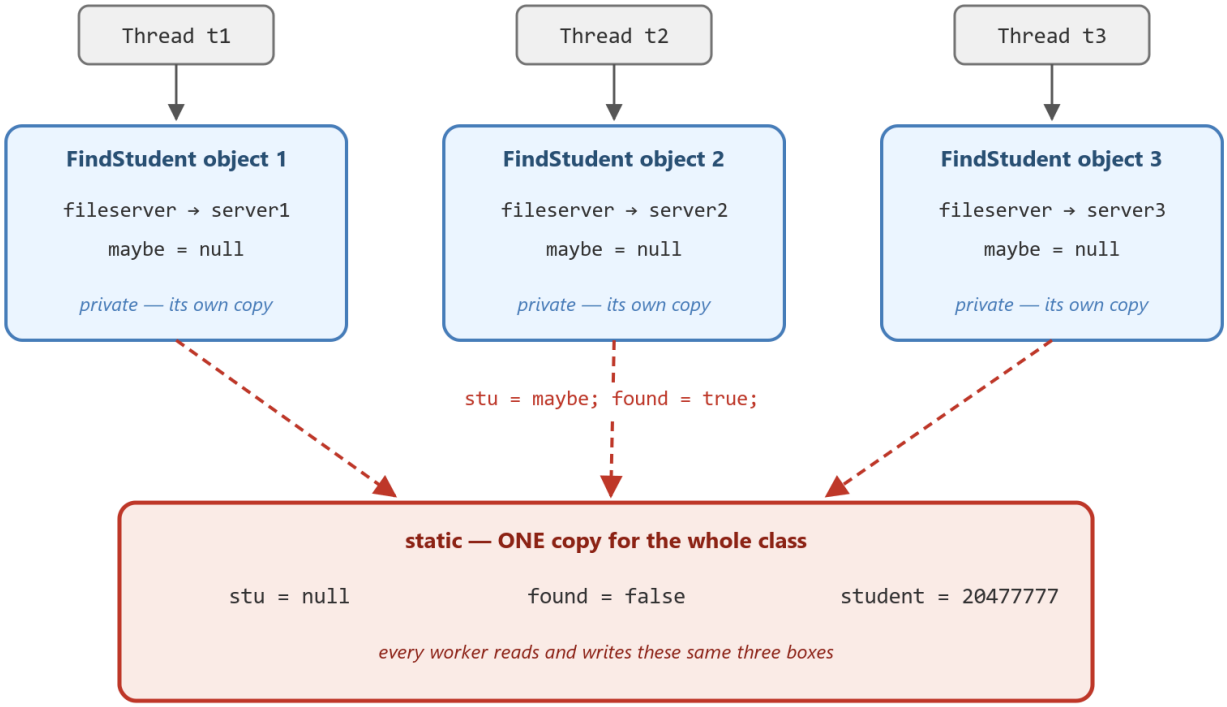
Given a PUID, find that student's record. It could be on any of the three, or on none of them.

- One thread would search server1, then server2, then server3. Three full scans happen one after another.
- Use concurrency: **one worker per server**, all three scanning at once. Whoever finds the student reports the answer.

Each `FindStudent` worker privately owns the server it scans.

But how do three workers report back?

A tempting design: through `static` fields.



```
private static Student stu = null;         // shared
private static String major = null;
private static boolean found = false;      // shared

public void run() {
    ... scan my server ...
    if (maybe.getID() == student) {
        stu = maybe;                       // three threads
        major = maybe.getMajor();          // can be here
        found = true;                      // at the same time
    }
}
```

Pause and predict (poll time)

Q. We spent today learning why something here is dangerous. What is it?

- **A.** Three threads may write `stu`, `major` and `found` at the same time, so the three fields can end up describing different records
- **B.** We might end up with the second match found, not the first
- **C.** Nothing: each thread scans its own server, so they never touch the same data
- **D.** `join()` is missing, so `main` reads too early

Find the Student, Fixed

Find the Student

The danger is that two different threads may find conflicting Student records, and we could end up in a situation where

- Thread1 writes `stu` last
- Thread2 writes `major` last

and our result is inconsistent.

Here's the code with synchronization of shared static fields:

```
class Main {
    void main() throws InterruptedException {
        int[] server1 = new int[500_000];
        int[] server2 = new int[500_000];
        int[] server3 = new int[500_000];
        for (int i = 0; i < 500_000; i++) {        // fake PUIDs
            server1[i] = 10_000_000 + i;
            server2[i] = 20_000_000 + i;
            server3[i] = 30_000_000 + i;
        }
        search(server1, server2, server3, 20_477_777);   // lives on server2
        search(server1, server2, server3, 99_999_999);   // not found
    }

    void search(int[] s1, int[] s2, int[] s3, int puid) throws InterruptedException {
        FindStudent.result = null;
        Thread t1 = new Thread(new FindStudent(s1, "server1", puid));
        Thread t2 = new Thread(new FindStudent(s2, "server2", puid));
        Thread t3 = new Thread(new FindStudent(s3, "server3", puid));
        t1.start(); t2.start(); t3.start();
        t1.join(); t2.join(); t3.join();
        if (FindStudent.result != null) {
            IO.println(puid + ": found on " + FindStudent.result);
        } else {
            IO.println(puid + ": no such student anywhere");
        }
    }
}
```

```
class FindStudent implements Runnable {
    static String result = null;                    // shared: guard it
    static Object gateKeeper = new Object();

    private int[] server;
    private String serverName;
    private int target;

    FindStudent(int[] server, String serverName, int target) {
        this.server = server;
        this.serverName = serverName;
        this.target = target;
    }

    @Override
    public void run() {
        for (int i = 0; i < server.length; i++) {
            if (server[i] == target) {
                synchronized (gateKeeper) {
                    if (result == null) {           // the first finder wins
                        result = serverName + " (record " + i + ")";
                    }
                }
                return;
            }
        }
    }
}
```

Remember:

- own what you can,
- guard what you must.

Decomposition

Two ways to carve up the work

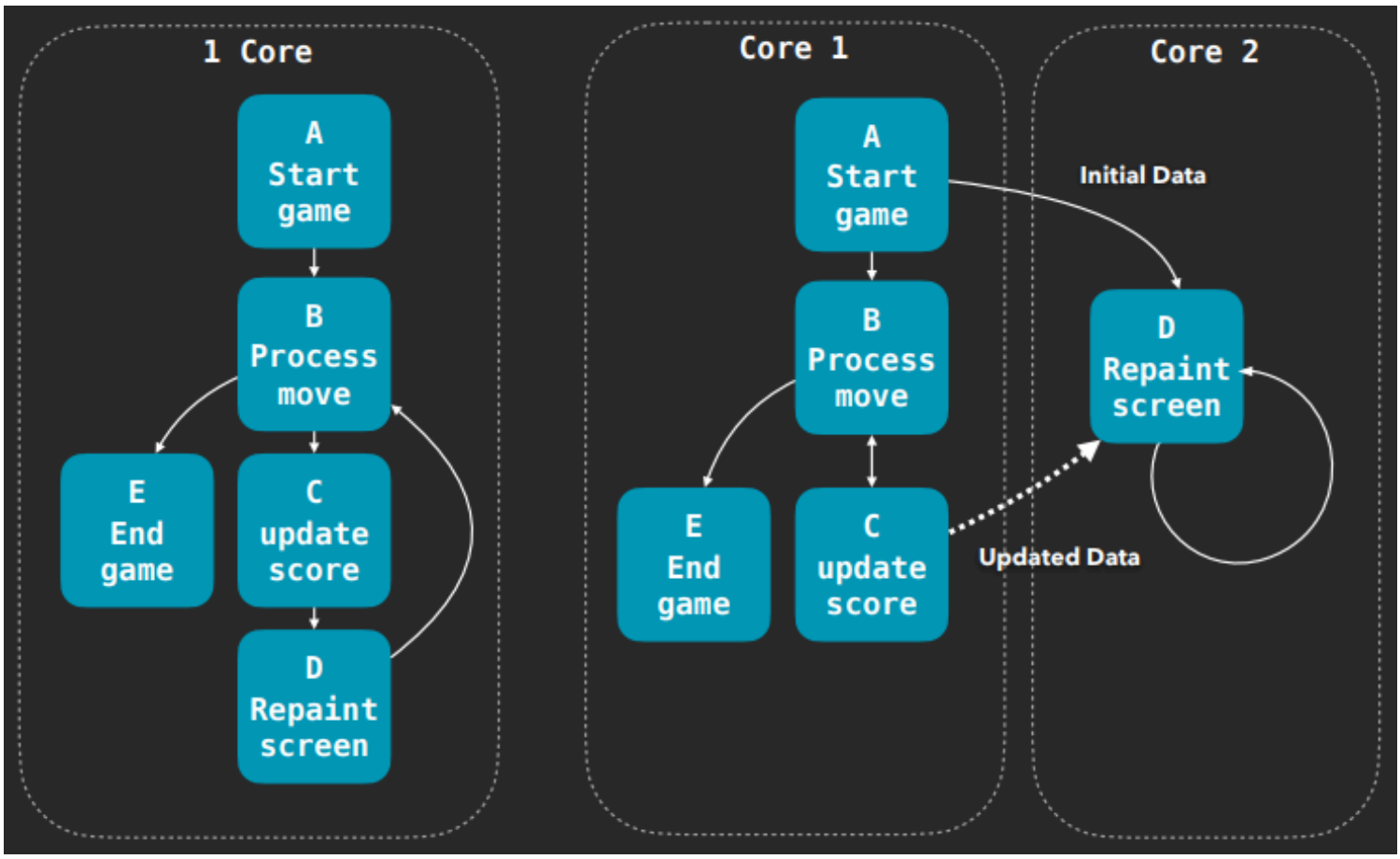
In order to use multiple cores effectively, we have to give them both some of the work.

How do you break down a large problem into pieces?

We call this *decomposing* the problem

Two approaches to decomposition:

Task decomposition: different *jobs* (different code) to different workers:



The game logic and the screen repainting run as separate tasks.

Benefit: the game stays **responsive** even when the logic is busy.

You can apply this in your own projects:

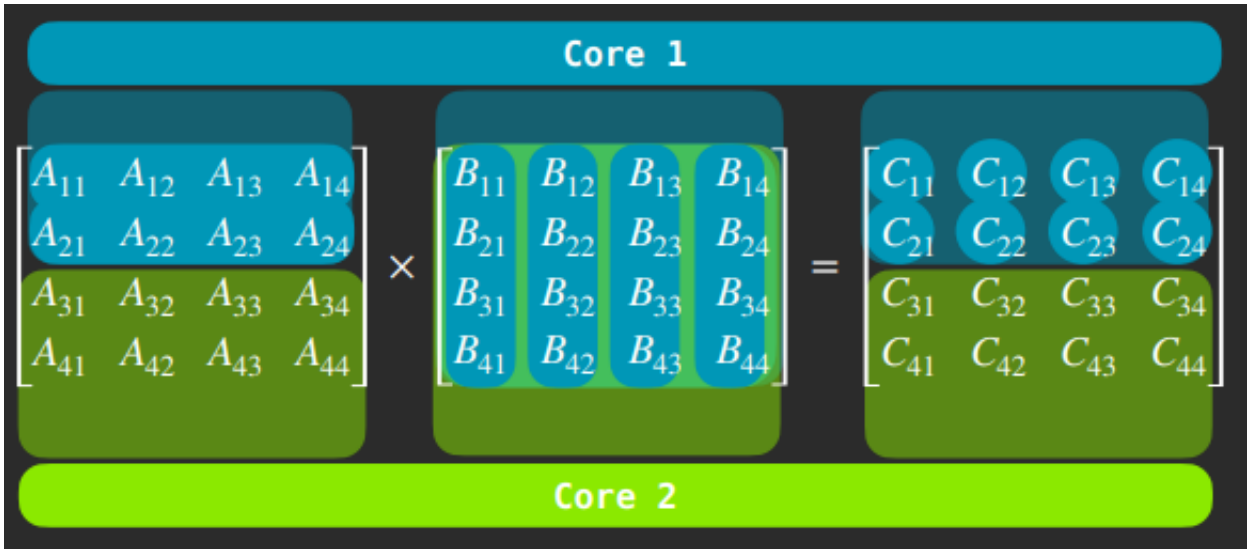
- Think of splitting up by the *kind of work* to be done
- Often, different kinds of work can be done independently

- The primary benefit is responsiveness

Domain decomposition: same job, different *slices of the data*:

Matrix Multiplication

- None of the entries of the matrix product depend on other parts of the answer.
- Two cores can compute half of the product at the same time.
- We partition the first input A into two halves.



Each subtask runs the

- *same code* but
- *on different input*

Each subdomain is given to a task running on a different core, if available

Primary benefit: raw speed

Primary drawback: not always possible, may require a deep understanding of our problem

Every worker runs the **same code** on its own slice.

Payoff: **raw speed,** which is a claim we should test.

Measuring the Speedup

Measuring the Speedup: Counting Primes

I can't claim that some approach gives you raw speed without some evidence.

Let's try to measure how long it takes to count all of the prime numbers up to some point.

Our thread task will be to check if numbers are prime:

```
public class Speedup implements Runnable {
    private int lo, hi;
    private int count = 0;                    // each worker's PRIVATE tally

    public Speedup(int lo, int hi) { this.lo = lo; this.hi = hi; }

    public void run() {
        for (int n = lo; n < hi; n++)
            if (isPrime(n)) count++;
    }

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int d = 2; d * d <= n; d++)
            if (n % d == 0) return false;
        return true;
    }
    ......
```

Let's setup a test.

First, a solo worker on the whole range:

```
int LIMIT = 8_000_000;

// --- one worker does everything ---
long start = System.currentTimeMillis();
Speedup solo = new Speedup(0, LIMIT);
solo.run();                               // just a method call, one thread
long soloMs = System.currentTimeMillis() - start;
```

Now let's do it again with four threads:

```
        Speedup[] workers = new Speedup[4];
        Thread[] threads = new Thread[4];
        int chunk = LIMIT / 4;
        for (int i = 0; i < 4; i++) {
            workers[i] = new Speedup(i * chunk, (i + 1) * chunk);
            threads[i] = new Thread(workers[i]);
            threads[i].start();
        }
        for (int i = 0; i < 4; i++)
            threads[i].join();
```

Let's put it all together:

```
class Main {
    void main() throws InterruptedException {
        int LIMIT = 80_000_000;

        // --- one worker does everything ---
        long start = System.currentTimeMillis();
        Speedup solo = new Speedup(0, LIMIT);
        solo.run();                                  // just a method call: one thread
        long soloMs = System.currentTimeMillis() - start;
        IO.println("1 thread:  " + solo.count + " primes in " + soloMs + " ms");

        // --- four workers, a quarter of the range each ---
        start = System.currentTimeMillis();
        Speedup[] workers = new Speedup[4];
        Thread[] threads = new Thread[4];
        int chunk = LIMIT / 4;
        for (int i = 0; i < 4; i++) {
            workers[i] = new Speedup(i * chunk, (i + 1) * chunk);
            threads[i] = new Thread(workers[i]);
            threads[i].start();
        }
        int total = 0;
        for (int i = 0; i < 4; i++) {
            threads[i].join();
            total += workers[i].count;
        }
        long fourMs = System.currentTimeMillis() - start;
        IO.println("4 threads: " + total + " primes in " + fourMs + " ms");
        IO.println(String.format("speedup: %.1fx", (double) soloMs / fourMs));
    }
}
```

```
class Speedup implements Runnable {
    private int lo, hi;
    int count = 0;                                  // each worker's PRIVATE tally

    Speedup(int lo, int hi) { this.lo = lo; this.hi = hi; }

    @Override
    public void run() {
        for (int n = lo; n < hi; n++) {
            if (isPrime(n)) count++;
        }
    }

    static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int d = 2; d * d <= n; d++) {
            if (n % d == 0) return false;
        }
        return true;
    }
}
```

Example output:

```
1 thread:  539777 primes in 4616 ms
4 threads: 539777 primes in 2042 ms
speedup: 2.3x
```

Why is this not 4x faster?

On Ed, we get roughly 2x faster. Why?

- The workload isn't evenly distributed.
  - The largest 1/4 of numbers are harder than the smallest 1/4.
  - We could distribute it more fairly, but it's not trivial.
- Hardware factors: four busy cores have a slower clockspeed than one busy core.
- Machine limitations: The Ed machine may have few cores available or a high load.

Take Home: Counting Up and Down

`Main.java` is given: two threads call `counter.up()` a million times each, two call `counter.down()` a million times each, all on one `Counter`, and `main` waits for all four. The count should end at exactly `0`. Press **Run** a few times: it doesn't.

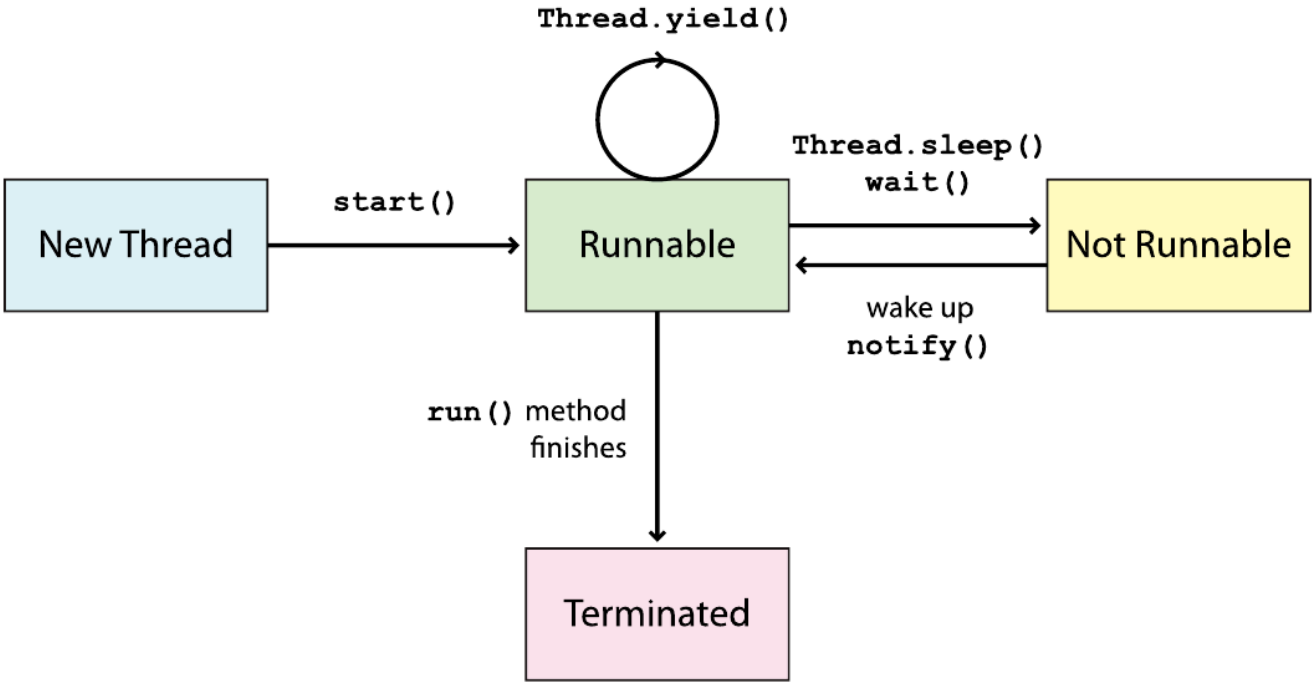
Fix `Counter.java`, and only `Counter.java`, so that the count is always exactly right. The Counter needs a gate that every thread using *this* Counter shares, and every touch of `count`, up, down, and reading it, must go through that gate. The usual rules apply: one dedicated gate object, the smallest block that covers the whole read-add-write, and nothing more.

Press **Test** to run the tests.

Extra Reading: A Thread's Life

Extra reading: not presented in lecture.

Reference: a thread's life



- New → `start()` → Runnable (available to run; the scheduler decides when)
- `sleep()` / `join()` park a thread in Not Runnable until it's time to continue
- `run()` finishes → Terminated

The `wait()`/`notify()` arrows on the figure are beyond the scope of CS 180.

`yield()` is obsolete and we recommend you never use it.

Extra Reading: The Modern Doors

Extra reading: not presented in lecture. Lambdas will be covered properly later.

Reference: the modern doors

Modern Java style has evolved.

You'll meet these in the real world:

```
Runnable task = () -> System.out.println("lambda!");   // a lambda IS a Runnable. () m

Thread v = Thread.startVirtualThread(task);            // virtual threads: JDK 21+
```

- A **lambda** is shorthand for the one-method `Runnable` class you now write by hand: same contract, less ceremony
- **Virtual threads** make threads cheap enough to have *millions* (one per web request)
- **Executors / thread pools** hire and manage the workers for you

The concepts we learned this week (`run`, `start`, `join`, synchronized sharing) are exactly what all three are shorthand for.

Example:

```
class Main {
    void main() throws InterruptedException {
        Runnable task = () -> IO.println("lambda on " + Thread.currentThread().getName
        Thread t = new Thread(task);
        t.start();
        t.join();

        Thread v = Thread.startVirtualThread(() -> IO.println("virtual on " + Thread.c
        v.join();
    }
}
```

We want you to be able to recognize these if you see them in the wild, and look them up for more information.